Karaoke Inteligente

Manual Técnico — IPC2, Universidad de San Carlos de Guatemala
carné 202307900

1. Arquitectura general

El programa está escrito en Python usando Toga (interfaz gráfica) y Briefcase (empaquetado), lo que permite compartir el 100% del código de lógica entre Windows y Android; solo cambian los módulos que hablan directamente con el sistema operativo.

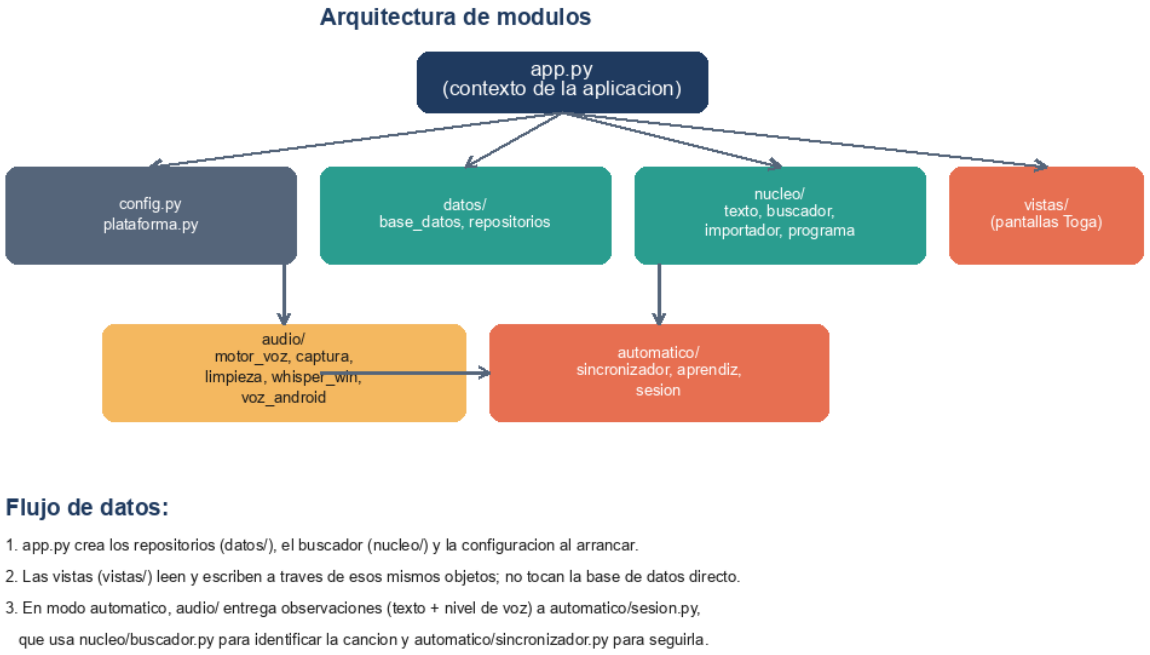


Diagrama 1. Módulos del programa y cómo se relacionan.

Resumen de cada módulo:

- **app.py** — punto de entrada; actúa como "contexto": guarda la conexión a la base de datos, los repositorios, el buscador y la configuración, y los pasa a cada pantalla.
- **config.py** — valores por defecto (ver sección 4) y la clase Configuracion que los lee/escribe.
- **plataforma.py** — todo lo que depende del sistema operativo: detectar si es Android, el selector de archivos (diálogo nativo en Windows, Intent en Android) y el permiso de micrófono.
- **datos/** — creación/detección de la base SQLite y un repositorio por tabla (canciones, programa, configuración, temporales).

- **nucleo/** — lógica que no depende de la interfaz: normalización de texto, el importador de archivos, el buscador (manual y automático) y el manejo del programa del cantante.
- **audio/** — motores de voz (interfaz común en motor_voz.py; implementaciones en whisper_win.py, voz_android.py y un motor simulado para pruebas), captura de micrófono y limpieza de audio.
- **automatico/** — el sincronizador (máquina de estados, ver sección 3), el aprendiz (captura canciones nuevas) y la sesión (une motor + sincronizador + programa).
- **vistas/** — una clase por pantalla, todas heredando de una base común con utilidades compartidas (botones, diálogos, paginación).

2. Modelo de datos

La base de datos es SQLite (un solo archivo, `CANCIONES.db`), guardado en la carpeta de datos propia de la app en cada plataforma. El programa detecta si el archivo existe al arrancar; si no, crea todas las tablas automáticamente.

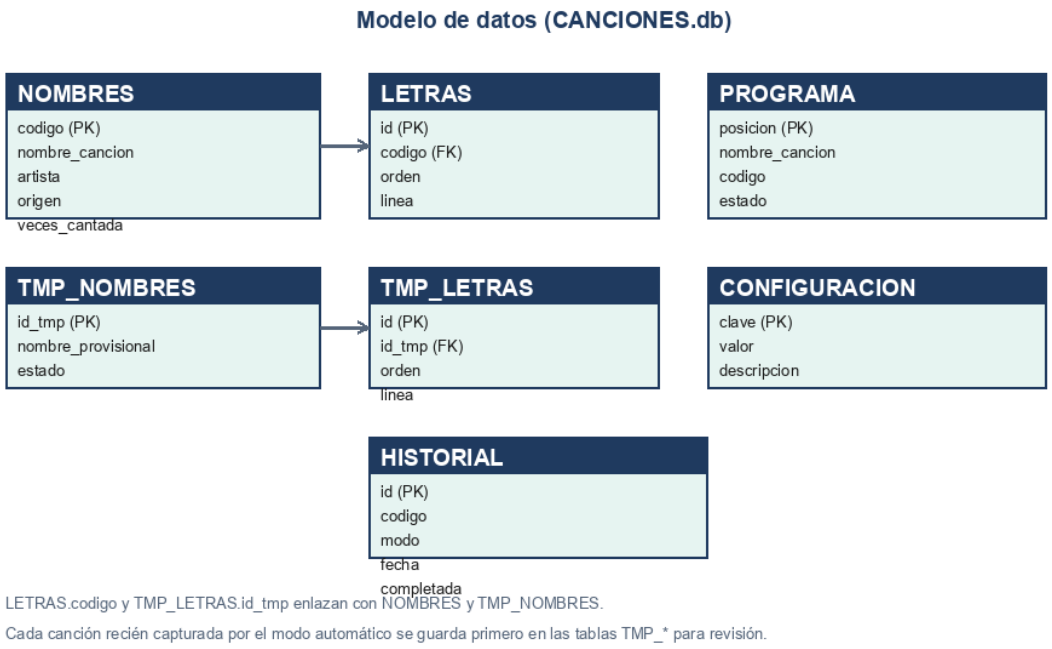


Diagrama 2. Tablas principales y sus campos.

Campos agregados sobre el mínimo del enunciado, y por qué:

- **NOMBRES.artista** — opcional; permite mostrarlo en la ficha de la canción sin forzarlo en la captura manual ni en el formato de archivo de texto.
- **NOMBRES.origen** — distingue canciones cargadas a mano/por archivo de las que entraron por el modo automático ("automatico"), útil para auditoría.
- **NOMBRES.veces_cantada** — contador simple para mostrar estadísticas básicas.
- **PROGRAMA** (tabla completa, no pedida explícitamente con ese nombre de tabla) — guarda el orden tentativo de canciones a cantar, con una bandera de estado (pendiente/cantada/saltada) para que el orden sea una sugerencia y no una cola rígida.
- **TMP_NOMBRES / TMP_LETRAS** — tablas temporales donde el modo automático escribe una canción mientras la está "aprendiendo"; solo pasan a NOMBRES/LETRAS cuando el operador las revisa y confirma el nombre real desde la pantalla (v) Revisar canciones capturadas.
- **CONFIGURACION** — tabla clave/valor para que los parámetros ajustables (sección 4) sobrevivan a cierres de la aplicación y sean editables desde (k) Configuración sin tocar código.
- **HISTORIAL** — una fila por cada vez que se canta una canción (modo y si se completó), para futuras estadísticas y para no perder esa información cuando el programa del cantante se reinicia.

3. Cómo se resolvió el modo automático

Esta es la parte de mayor complejidad del enunciado: identificar qué canción se está cantando y en qué línea va, a partir de audio con ruido y de una transcripción imperfecta, sin que el cantante tenga que detenerse ni esperar.



Diagrama 3. Flujo de datos del modo automático, de principio a fin.

Limpieza del audio (Windows)

Antes de transcribir se aplican, en orden: resta del nivel de continua (DC), un filtro pasa-banda FIR (80–4000 Hz, la banda de la voz humana) construido por ventana-sinc, resta espectral contra un perfil de ruido de fondo grabado al inicio, y normalización de volumen. Se eligió un filtro FIR por ventana-sinc en vez de recortar directamente el espectro de todo el bloque de audio (un enfoque más simple) porque ese recorte directo introduce distorsión en el tiempo que el motor de voz a texto confunde con fonemas reales, empeorando la transcripción con ruido de fondo.

Detección de voz (VAD)

En Windows se compara la energía de la señal contra un piso de ruido calibrado en los primeros segundos de silencio. En Android se usa el nivel RMS que reporta el propio reconocedor del sistema (onRmsChanged), pero en vez de un umbral fijo —que no se ajustaba igual de bien a distintos micrófonos— se seguie el piso de ruido con un rastreador asimétrico: baja rápido al detectar silencio más bajo y sube despacio, para no "perseguir" la propia voz del cantante.

Identificación de la canción (buscador)

Cada fragmento transcrito se compara contra todas las ventanas posibles de cada canción candidata (solo se consideran candidatas las canciones que comparten al menos 2 palabras de 3+ letras con lo oído, para no comparar contra toda la base). El puntaje de cada ventana combina dos señales a partes iguales: el parecido de caracteres (SequenceMatcher, reescalado porque dos textos sin relación ya dan ~0.5 de parecido) y la fracción de palabras oídas que aparecen en esa ventana (tolerando errores de transcripción con un parecido >= 0.8 por palabra). Se reconoce una canción cuando el mejor puntaje supera un umbral y, o bien saca una ventaja clara sobre la segunda opción, o ya era la candidata en la observación anterior (evita cambiar de opinión por una sola observación ruidosa).

Ese umbral se ajustó con datos, no a ojo: se simularon cientos de fragmentos con errores típicos de un reconocedor de voz (sustituciones, palabras perdidas, palabras de más) sobre las canciones de prueba. Con el umbral original, el puntaje promedio de una identificación correcta bajo ruido fuerte caía justo debajo del corte; se bajó el umbral y el acierto subió sin generar ningún falso positivo contra texto sin

relación con las canciones (probado también con la simulación).

Seguimiento de la línea (predictor-corrector)

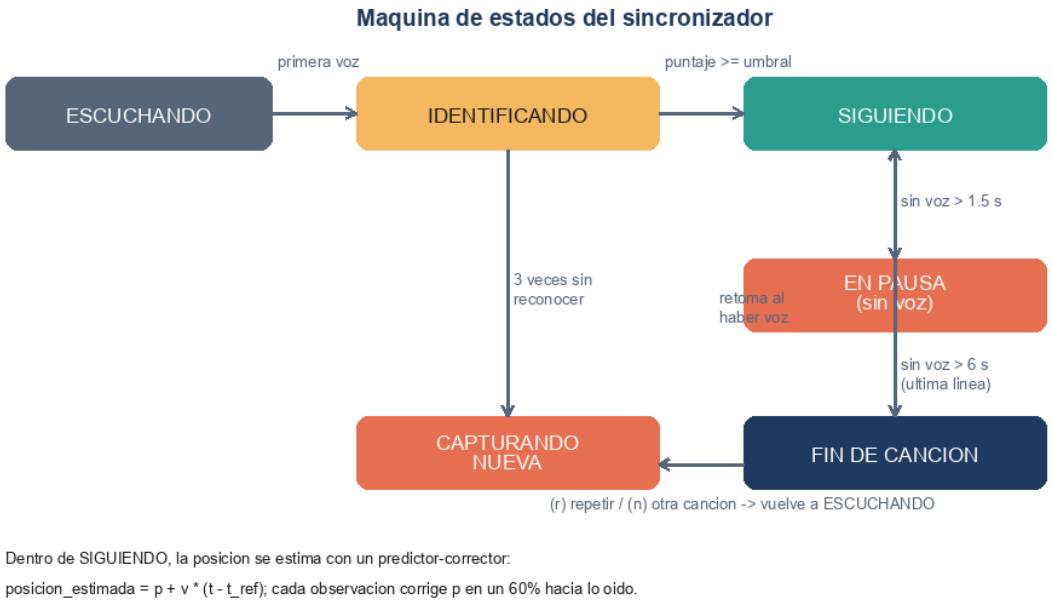


Diagrama 4. Estados del sincronizador.

Una vez identificada la canción, la posición (en palabras) se modela como `p■(t) = p + v·(t − t_ref)`: entre observaciones la letra avanza sola a una velocidad estimada v (palabras por segundo), así la pantalla no se queda atrás aunque el motor de voz tarde en responder. Cada observación nueva corrige la posición acercándola un 60% hacia lo que realmente se oyó, y ajusta v con un promedio exponencial. Si la posición oída está muy lejos de la esperada (el cantante repitió una línea o se saltó una), el salto solo se acepta cuando hay certeza (puntaje muy alto, o dos observaciones seguidas apuntando al mismo lugar). Sin voz por más de un tiempo configurable la predicción se congela (estado "en pausa"), y sin voz en la última línea por más tiempo aún se da la canción por terminada.

4. Supuestos hechos y cómo se resolvieron

- **Asignación de teclas** — el enunciado deja ambigua la tecla para "ignorar" y para "línea igual". Se asignó (i) a ignorar (coincide con la letra inicial de la palabra) y (l) a línea igual, dejando (s) para "seguir" después de ignorar, tal como describe el enunciado.
- **Código de canción** — se genera automáticamente como un entero autoincremental al insertar en NOMBRES; nunca lo escribe el usuario.
- **Canción "nueva" vs. conocida (modo automático)** — si durante 3 observaciones seguidas ninguna canción conocida alcanza ni siquiera el umbral más bajo (UMBRAL_NUEVA) y el fragmento tiene suficiente variedad de palabras, se asume que es una canción que no está en la base y se empieza a capturar en las tablas TMP_*.
- **Formato del archivo de texto para alimentar (t)** — nombre de la canción en una línea, la letra debajo (una línea de letra por renglón), y una línea en blanco entre canciones; es el formato más simple que cubre "nombre + letra" sin exigir marcado adicional.
- **Tamaño de página ("de 10 en 10")** — se usa el mismo valor configurable (LINEAS_POR_PAGINA) tanto para el listado de canciones (w) como para los grupos de líneas al ver una letra (x), ya que el enunciado pide "de 10 en 10" en ambos casos.
- **Caracteres promedio por línea (80) y máximo (120)** — controlan el ancho estimado de la vista y el punto en que una línea muy larga se parte en dos al importar o capturar texto; ambos se pueden cambiar desde (k) Configuración (CARACTERES_PROMEDIO_LINEA y MAX_CARACTERES_LINEA) sin tocar código, para adaptarse a canciones con versos más largos o más cortos que el promedio del enunciado.
- **Motor de voz "auto"** — por defecto el programa elige Whisper en Windows y el reconocedor del sistema en Android según la plataforma detectada, pero se puede forzar un motor específico (incluido uno "simulado", donde el operador escribe lo que "se escuchó", útil para demostrar y probar la sincronización sin micrófono).

5. Cómo poner a funcionar el programa

Opción A — instalar los binarios ya compilados (más rápido):

Descargue el instalador de Windows (.msi) o Android (.apk) desde el Release v1.0.0 del repositorio: https://github.com/jeansitox5/IPC2_KaraokeInteligente_202602_202307900/releases/tag/v1.0.0. El instalador de Windows puede mostrar la advertencia de SmartScreen por no tener un certificado comercial pagado ("Más información" → "Ejecutar de todas formas"); el .apk requiere activar "Instalar apps desconocidas" para el navegador o gestor de archivos usado.

Opción B — correr desde el código fuente:

Requiere Python 3.13 instalado.

```
git clone
https://github.com/jeansitox5/IPC2_KaraokeInteligente_202602_202307900.git
cd IPC2_KaraokeInteligente_202602_202307900
python -m venv venv
venv\Scripts\activate
pip install briefcase
briefcase dev -r ■# ejecuta la app con la interfaz de Windows
briefcase dev --test ■# corre las 21 pruebas automatizadas
```

Para generar los instaladores desde el código: `briefcase create windows` + `briefcase package windows --adhoc-sign` (Windows), o `briefcase create android` + `briefcase package android -p debug-apk` (Android, requiere el SDK de Android y un dispositivo o emulador conectado para probar con `briefcase run android`).

Whisper (solo Windows)

El modo automático en Windows necesita `whisper-cli.exe` y un modelo (.bin) dentro de `src/karaoke/resources/whisper/`. El script `scripts/descargar_whisper.ps1` descarga ambos automáticamente. En Android no se necesita: se usa el reconocedor de voz propio del sistema operativo.

6. Estructura de carpetas

```
IPC2_KaraokeInteligente_202602_202307900/
  src/karaoke/
    app.py              - punto de entrada / contexto de la app
    config.py           - valores por defecto configurables
    plataforma.py       - diferencias Windows/Android
    datos/              - base de datos y repositorios
    nucleo/             - texto, buscador, importador, programa
    audio/              - motores de voz, captura, limpieza
    automatico/         - sincronizador, aprendiz, sesion
    vistas/             - pantallas (interfaz)
    resources/          - iconos, ejemplos, whisper (solo Windows)
  tests/                - pruebas automatizadas (pytest)
  scripts/              - utilidades (descarga de Whisper)
  docs/                 - manuales y guía de entrega
  pyproject.toml        - configuración de Briefcase
```

7. Decisiones de diseño

- **Toga + Briefcase** — una sola base de código Python para Windows y Android, en vez de mantener dos interfaces separadas; el costo es algo menos de control fino sobre el layout nativo de cada plataforma.
- **SQLite** — motor embebido, sin servidor que instalar ni configurar, soportado de forma nativa por Python en ambas plataformas, y suficiente para el volumen de datos de este programa.
- **Whisper en Windows** — funciona completamente sin conexión a internet, sin costo por uso y con buena precisión en español; se usa la variante whisper.cpp por ser liviana y no requerir GPU.
- **Reconocedor nativo en Android** — evita distribuir un modelo de reconocimiento de voz dentro de la app (aumentaría mucho el tamaño del .apk) y aprovecha el reconocimiento ya optimizado del sistema operativo, con soporte de resultados parciales en tiempo real.